



DS & OOP (EC-204)

DE-44 Mechatronics

Syndicate – C

Project Report

Title: Uno Game Implementation in C++ using Data Structures and Object-Oriented Programming.

Name of members:

1. NS Fatima Sarwar

Reg No #409661

2. NS Muhammad Ahmed Hussain

Reg No #427781

Submitted to: Dr. Tahir Nawaz

Table of Contents

•	Introduction:	4
-	Background:	4
-	Objective:	4
•	Game Rules:	4
-	Deck Composition:	4
-	Starting the Game:	4
-	Game Flow:	4
-	Winning the Game:	4
•	Motivation Behind Project Selection:	4
•	Overview of the Code:	5
-	UnoCard Class:	5
-	UnoDeck Class:	5
-	UnoGame Class:	5
-	Main Function:	5
•	UnoCard Class:	6
•	UnoDeck Class:	6
-	Member Variables:	6
-	initializeDeck Function:	6
-	addCardToDeck Function:	7
-	shuffle Function:	7
-	drawCard Function:	8
-	removeCard Function:	8
-	toString Function:	9
-	clearDeck Function:	9
-	swapNodes Function:	10
-	getNodeAtPosition Function:	10
•	UnoGame Class:	10
-	display Function:	10
-	start Function:	11
-	getTopCard Function:	11
-	drawInitialCards Function:	12
-	drawCard Function:	12
-	playCard Function:	13
-	validCardPlay Function:	14
-	handleSpecialCards Function:	14
-	handleSkip Function:	15

-	checkPlayerTurn Function:	15
-	colorToString and valueToString Functions:	15
•	Main Function (Compact Version):	16
}		16
Explanation:		16
•	Execution of the Program:	16
•	Difficulties encountered:	18
•	Algorithm Issues:	19
•	Conclusion:	19

- **Introduction:**

- **Background:**

Uno is a popular card game known for its simplicity and entertainment. This project aimed to implement a console-based Uno game in C++ to explore object-oriented programming principles and enhance coding skills.

- **Objective:**

The primary objective was to create an interactive Uno game where players could draw and play cards, following the standard Uno rules.

- **Game Rules:**

- **Deck Composition:**

- Consists of four colors: Red, Blue, Green, and Yellow.
- Each color has cards numbered from 0 to 9, along with special action cards (DrawTwo, Skip, Reverse).
- Wild cards include Wild and Wild_Four.

- **Starting the Game:**

- Each player is dealt seven cards at the beginning.
- The top card of the deck is revealed to initiate the discard pile.

- **Game Flow:**

- Players take turns playing a card that matches the color or number of the top card.
- Special cards trigger actions like skipping a player, reversing the game direction, or drawing additional cards.

- **Winning the Game:**

- A player wins when they have one card left and successfully play it.

- **Motivation Behind Project Selection:**

The motivation behind selecting the Uno card game as a project stems from the desire to apply and integrate the diverse set of data structures (DS) and object-oriented programming (OOP) concepts learned throughout this semester. Uno, being a card game, naturally involves the representation and manipulation of data, making it an ideal

candidate for exploring DS concepts such as linked lists for deck management and efficient card shuffling. Furthermore, Uno's rule complexity, which includes various card values and colors, turn-based gameplay, and special card effects, provides an excellent context for implementing OOP principles. Through this project, we aim to solidify our understanding of DS and OOP, honing our programming skills by creating a cohesive and functional Uno game that reflects the intricacies of the real-world card game.

- **Architecture of the Code:**

- **UnoCard Class:**

- **Enumerations:** Defined Color and Value enums representing card attributes.
- **Constructor:** Initializes UnoCard with a given color and value.
- **String Conversion:** stringToUnoCard function converts a string representation to UnoCard.
- **ToString Function:** Converts UnoCard to a string.

- **UnoDeck Class:**

- **Nested Node Class:** Represents a node in the UnoDeck linked list.
- **Initialization:** Creates a deck with standard Uno cards.
- **Shuffling:** Implements the Fisher-Yates shuffle algorithm.
- **Drawing Cards:** Draws a card from the deck.
- **Removing Cards:** Removes a specified card from the deck.
- **Resetting Deck:** Clears and initializes the deck.

- **UnoGame Class:**

- **Members:**
currentPlayer, playerIds, playerHand, numPlayers, deck, stockpile, discardPile, validColor, validValue, gameDirection.
- **Functions:**
 - **start:** Initiates the game, draws the initial card, and sets game conditions.
 - **display:** Displays the game state.
 - **drawInitialCards:** Draws initial cards for each player.
 - **drawCard:** Draws a card for the current player.
 - **playCard:** Plays a card, updating game conditions.
 - **validCardPlay:** Checks if a card play is valid.
 - **checkPlayerTurn:** Verifies if it's the correct player's turn.
 - **handleSkip:** Handles skipping a player.
 - **handleSpecialCards:** Handles special action cards.

- **Main Function:**

- **User Interaction:** Collects player names, initializes the UnoGame, and enters a loop for player moves.
- **Console Output:** Displays game information, handles player input, and manages

game flow.

- **UnoCard Class:**

The UnoCard class represents an individual Uno card. It employs enums for Color and Value to categorize cards, making the code more readable and reducing the likelihood of errors when referencing card attributes. For example:

```
enum Color { Red, Blue, Green, Yellow, Wild };
enum Value { Zero, One, Two, ..., Wild_Four };
```

Why Enums? Enums provide a convenient way to represent a fixed set of values. In this context, using enums for colors and values ensures clarity when working with Uno cards, making it easier to understand and maintain the code.

- **UnoDeck Class:**

- **Member Variables:**

```
class UnoDeck
{
private:
    class Node
    {
    public:
        UnoCard card;
        Node* next;

        Node (UnoCard c) : card(c), next(nullptr) {}
    };

    Node* cards;
    int cardsInDeck;
};
```

Explanation:

- Node: This is a private nested class representing a node in a linked list. Each node contains an UnoCard and a pointer to the next node.
- cards: A pointer to the first node in the linked list, representing the top of the deck.
- cardsInDeck: An integer tracking the number of cards in the deck.

Design Choice:

Using a linked list for the deck allows for efficient card removal and insertion operations. The Node class encapsulates the details of a single card node, promoting encapsulation and modular design.

- **initializeDeck Function:**

```
void UnoDeck::initializeDeck()
{
    UnoCard::Color colors[] = { UnoCard::Red, UnoCard::Blue, UnoCard::Green,
    UnoCard::Yellow };

    for (int i = 0; i < 4; ++i)
```

```

{
    UnoCard::Color color = colors[i];

    addCardToDeck(UnoCard(color, UnoCard::Zero));

    for (int j = 1; j < 10; j++)
    {
        addCardToDeck(UnoCard(color, static_cast<UnoCard::Value>(j)));
        addCardToDeck(UnoCard(color, static_cast<UnoCard::Value>(j)));
    }

    UnoCard::Value values[] = { UnoCard::DrawTwo, UnoCard::Skip,
    UnoCard::Reverse };

    for (UnoCard::Value value : values)
    {
        addCardToDeck(UnoCard(color, value));
        addCardToDeck(UnoCard(color, value));
    }
}
// ...
}

```

Explanation:

- The function populates the deck with standard Uno cards, including numbers, special cards, and their duplicates.
- It uses loops to iterate through colors, numbers, and special card values, adding corresponding cards to the deck using addCardToDeck function.

Design Choice:

The initializeDeck function follows Uno's standard deck composition. Using loops to iterate through colors and values allows for concise and readable code. The addCardToDeck function ensures modularity by encapsulating the addition of cards.

- addCardToDeck Function:

```

void UnoDeck::addCardToDeck(UnoCard card)
{
    Node* newNode = new Node(card);
    newNode->next = cards;
    cards = newNode;
    ++cardsInDeck;
}

```

Explanation:

- This function adds a new card to the top of the deck by creating a new node and updating pointers.
- The function ensures that the new card becomes the first in the linked list, representing the top of the deck.

Design Choice:

Adding cards to the front of the linked list ensures constant-time complexity for adding cards. This choice is made because Uno decks primarily involve drawing from the top, making the front insertion efficient.

- shuffle Function:

```

void UnoDeck::shuffle()
{

```

```

        srand(static_cast<unsigned int>(time(nullptr)));

        for (int i = cardsInDeck - 1; i > 0; --i)
        {
            int j = rand() % (i + 1);

            swapNodes(i, j);
        }
}

```

Explanation:

- The shuffle function employs the Fisher-Yates shuffle algorithm to randomize the order of cards in the deck.
- It uses rand() to generate random indices and swapNodes function to swap cards at those indices.

Design Choice:

- Fisher-Yates shuffle is a widely-used and unbiased shuffling algorithm. Using this algorithm ensures that the deck is thoroughly randomized, promoting fairness in the game.

- **drawCard Function:**

```

UnoCard drawCard()
{
    if (isEmpty())
    {
        throw runtime_error("Cannot draw a card since there are no cards
in the deck");
    }

    UnoCard drawnCard = cards->card;
    removeCard(cards->card);

    return drawnCard;
}

```

Explanation:

- drawCard removes the top card from the deck and returns it.
- It checks if the deck is empty and throws an exception if an attempt is made to draw from an empty deck.

Design Choice:

- Drawing a card involves removing the top card from the deck. This function encapsulates the logic of drawing a card and removes the card from the deck using the removeCard function.

- **removeCard Function:**

```

void removeCard(UnoCard card)
{
    if (isEmpty())
    {
        throw runtime_error("Cannot remove a card since there are no
cards in the deck");
    }

    if (cards->card.color == card.color && cards->card.value == card.value)
    {
        Node* temp = cards;
        cards = cards->next;
        delete temp;
        --cardsInDeck;
    }
    else

```

```

{
    Node* current = cards;
    Node* previous = nullptr;

    while (current != nullptr && !(current->card.color == card.color
&& current->card.value == card.value))
    {
        previous = current;
        current = current->next;
    }

    if (current == nullptr)
    {
        throw runtime_error("Card not found in the deck");
    }

    previous->next = current->next;
    delete current;
    --cardsInDeck;
}

```

Explanation:

- removeCard removes a specified card from the deck.
- It handles both the case where the card is at the top of the deck and when it's within the deck by traversing the linked list.

Design Choice:

- Removing a card involves searching for the card in the linked list. If the card is at the front of the deck, it can be removed directly. However, if it's elsewhere in the deck, additional logic is needed to locate and remove it. This design choice accommodates different scenarios efficiently.

- **toString Function:**

```

std::string toString() const
{
    std::string result;
    Node* current = cards;

    while (current != nullptr)
    {
        result += current->card.toString() + " , ";
        current = current->next;
    }

    return result;
}

```

Explanation:

- toString generates a string representation of the entire deck for debugging or display purposes.
- It traverses the linked list, concatenating each card's string representation to the result string.

- **clearDeck Function:**

```

void clearDeck()
{
    while (cards != nullptr)
    {
        Node* temp = cards;
        cards = cards->next;
        delete temp;
    }
}

```

```

        cardsInDeck = 0;
    }

```

Explanation:

- clearDeck deallocates memory for all nodes in the deck, effectively emptying the deck.
- It ensures that no memory leaks occur by deleting each node and updating the cardsInDeck count.

- swapNodes Function:

- This function is responsible for swapping two nodes in the linked list-based deck.
- It takes two indices `i` and `j` as parameters.
- Checks if `i` and `j` are different to avoid unnecessary swaps.
- Retrieves the nodes at positions `i` and `j` using the `getNodeAtPosition` function.
- Swaps the `UnoCard` values between the two nodes, effectively swapping the cards.

- getNodeAtPosition Function:

- This function retrieves the node at a specified position in the linked list.
- Takes an integer `position` as a parameter.
- Initializes a pointer `current` to the head of the linked list (`cards`).
- Uses a loop to traverse the linked list, moving `position` times to reach the desired node.
- Returns the pointer to the node at the specified position.

• UnoGame Class:

- display Function

```

void display()
{
    SetConsoleTextAttribute(h, 0x94); //9 for background color and $ for word
    color
    char a = 178;
    for (int i = 0; i < 76; i++)
    {
        cout << a;
    }
    cout << "UNO Game Started!";
    for (int i = 0; i < 75; i++)
    {
        cout << a;
    }
    cout << "\n\n";
}

```

Explanation:

- This function is responsible for displaying a formatted title at the start of the Uno game.
- It uses the Windows Console API (SetConsoleTextAttribute) to set the console text color.
- A character (a) is repeated to create a decorative border around the game title.

- **start Function:**

```
void start()
{
    //display();
    if (deck.isEmpty())
    {
        // Reset and shuffle the deck if it is empty
        deck.reset();
        deck.shuffle();
    }

    UnoCard card = deck.drawCard();
    validColor = card.getColor();
    validValue = card.getValue();

    if (card.getValue() == UnoCard::Wild_Card || card.getValue() ==
    UnoCard::Wild_Four || card.getValue() == UnoCard::DrawTwo)
    {
        start();
    }
    else if (card.getValue() == UnoCard::Skip)
    {
        SetConsoleTextAttribute(h, 0x94);
        cout << playerIds[currentPlayer] << " was skipped\n" << endl;
        handleSkip();
    }
    else if (card.getValue() == UnoCard::Reverse)
    {
        SetConsoleTextAttribute(h, 0x94);
        cout << playerIds[currentPlayer] << " changed the game direction\n" <<
endl;
        gameDirection ^= true;
        currentPlayer = numPlayers - 1;
    }

    stockpile.addCardToDeck(card);
}
```

Explanation:

-
- This function starts the Uno game by initializing the deck and drawing the first card.
- It checks for special starting cards (Wild, Wild_Four, DrawTwo, Skip, Reverse) and handles them accordingly.
- If a Skip or Reverse card is drawn, it informs the players and adjusts the turn accordingly.

- **getTopCard Function:**

```
UnoCard getTopCard()
{
    return stockpile.drawCard();
}
```

Explanation:

- This function retrieves and returns the top card from the stockpile.
- It utilizes the UnoDeck::drawCard function to ensure the stockpile is not empty.

- **drawInitialCards Function:**

```
void drawInitialCards(UnDeck& hand)
{
    for (int i = 0; i < 7; ++i)
    {
        hand.addCardToDeck(deck.drawCard());
    }
}
```

Explanation:

- This function is called during game initialization to draw seven cards for each player.
- It adds cards from the deck to the player's hand using the `UnDeck::addCardToDeck` function.

- **drawCard Function:**

```
void drawCard(string pid)
{
    checkPlayerTurn(pid);

    if (deck.isEmpty())
    {
        deck.reset();
        deck.shuffle();
    }

    // Check again if the deck is still empty after reset and shuffle
    if (deck.isEmpty())
    {
        cerr << "Error: Cannot draw a card since there are no cards in the deck
even after reset and shuffle." << endl;
        exit(EXIT_FAILURE);
    }

    playerHand[currentPlayer].addCardToDeck(deck.drawCard());

    if (gameDirection == false)
    {
        currentPlayer = (currentPlayer + 1) % numPlayers;
    }
    else
    {
        currentPlayer = (currentPlayer - 1 + numPlayers) % numPlayers;
    }
}
```

Explanation:

- This function allows a player to draw a card from the deck.
- It checks if the deck is empty, and if so, resets and shuffles it.
- The drawn card is added to the player's hand, and the next player's turn is determined based on the game direction.

- playCard Function:

```
void playCard(string pid, UnoCard card, UnoCard::Color declaredColor)
{
    checkPlayerTurn(pid);

    UnoDeck& hand = playerHand[currentPlayer];

    if (!validCardPlay(card))
    {
        if (card.getColor() == UnoCard::Wild)
        {
            validColor = card.getColor();
            validValue = card.getValue();
        }
        else if (card.getColor() != validColor)
        {
            cout << "Invalid player move, expected color: " <<
colorToString(validColor)
                << " but got color " << colorToString(card.getColor()) <<
endl;
            throw runtime_error("Invalid color submission");
        }
        else if (card.getValue() != validValue)
        {
            cout << "Invalid player move, expected value: " <<
valueToString(validValue)
                << " but got value " << valueToString(card.getValue()) <<
endl;
            throw runtime_error("Invalid value submission");
        }
    }

    hand.removeCard(card);

    if (hand.isEmpty())
    {
        SetConsoleTextAttribute(h, 0x94);
        cout << playerIds[currentPlayer] << "\n won the game!\n" << endl;
        exit(0);
    }

    validColor = card.getColor();
    validValue = card.getValue();
    stockpile.addCardToDeck(card);

    if (gameDirection == false)
    {
        currentPlayer = (currentPlayer + 1) % numPlayers;
    }
    else
    {
        currentPlayer = (currentPlayer - 1 + numPlayers) % numPlayers;
    }

    if (card.getColor() == UnoCard::Color::Wild)
    {
        validColor = declaredColor;
    }

    handleSpecialCards(card);
}
```

Explanation:

- This function is responsible for a player playing a card in the game.
- It checks if the played card is a valid move based on the current game state.
- If the card is a Wild card, it uses the declared color; otherwise, it checks the color and value.
- The played card is removed from the player's hand, and the game state is updated.
- If the player's hand is empty, they win the game.
- The function also handles setting the declared color for Wild cards and special card effects.

- validCardPlay Function:

```
bool validCardPlay(UnoCard card) const
{
    if (card.getColor() == validColor || card.getValue() == validValue)
        return true;
    else
        return false;
}
```

Explanation:

- This function checks if playing a given card is a valid move based on the current game state.
- It returns true if the card's color matches the valid color or its value matches the valid value, indicating a valid move.
- Otherwise, it returns false.

- handleSpecialCards Function:

```
void handleSpecialCards(UnoCard card)
{
    if (card.getValue() == UnoCard::DrawTwo)
    {
        drawCards(playerIds[currentPlayer], 2);
    }
    else if (card.getValue() == UnoCard::Wild_Four)
    {
        drawCards(playerIds[currentPlayer], 4);
    }
    else if (card.getValue() == UnoCard::Skip)
    {
        SetConsoleTextAttribute(h, 0x94);
        cout << playerIds[currentPlayer] << " was skipped!\n" << endl;
        handleSkip();
    }
    else if (card.getValue() == UnoCard::Reverse)
    {
        SetConsoleTextAttribute(h, 0x94);
        cout << playerIds[currentPlayer] << " changed the game direction!\n" << endl;
        gameDirection ^= true;
        currentPlayer = (currentPlayer - 2 + numPlayers) % numPlayers;
    }
}
```

```
}  
}
```

Explanation:

- This function is responsible for handling the effects of special cards.
- It uses a switch statement based on the card's value to determine the specific action to take.
- For Skip, it calls `handleSkip`; for Reverse, it calls `handleReverse`; for DrawTwo, it calls `handleDrawTwo`; and for Wild cards, it calls `handleWildCard`.

- **handleSkip Function:**

```
void handleSkip()  
{  
    if (gameDirection == false)  
    {  
        currentPlayer = (currentPlayer + 1) % numPlayers;  
    }  
    else  
    {  
        currentPlayer = (currentPlayer - 1 + numPlayers) % numPlayers;  
    }  
}
```

Explanation:

- This function handles the Skip card effect.
- It skips the next player's turn based on the current game direction.

- **checkPlayerTurn Function:**

```
void checkPlayerTurn(string pid) const  
{  
    if (playerIds[currentPlayer] != pid)  
    {  
        throw runtime_error("It is not " + pid + "'s turn");  
    }  
}
```

Explanation:

- This function checks if it is the turn of the specified player.
- If the provided player ID does not match the current player's ID, an error is displayed, and the program exits.

- **colorToString and valueToString Functions:**

Explanation:

- These functions convert `UnoCard::Color` and `UnoCard::Value` enum values to string representations.
- They are used for displaying meaningful information in console messages.

- **Main Function (Compact Version):**

```
int main()
{
    UnoGame unoGame;

    unoGame.display();
    unoGame.initializeGame();

    while (true)
    {
        unoGame.displayCurrentState();
        unoGame.displayPlayerHand();

        unoGame.playerMove();

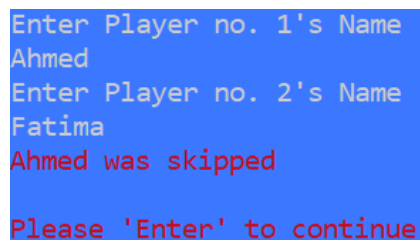
        if (unoGame.isGameOver())
        {
            break;
        }
    }

    return 0;
}
```

Explanation:

- The program initializes an instance of the UnoGame class and displays the game title.
- The game is initialized, including shuffling the deck and distributing cards.
- The core game loop begins, where the current state of the game, the player's hand, and the player's move are displayed iteratively.
- The loop continues until the game is over, at which point the program exits.
- This int main() function provides the structure for the Uno game, ensuring continuous gameplay until a player wins or the game is manually terminated.

- **Execution of the Program:**



```
Enter Player no. 1's Name
Ahmed
Enter Player no. 2's Name
Fatima
Ahmed was skipped

Please 'Enter' to continue
```

Figure 1 Output (Entering Player Names)

- The console prompts the user to input the name of the players as desired.
- The top card of the deck was a Skip Card so first player's turn was skipped and the game was started by the second player.

```

UNO Game Started!

Current Player: Fatima

Top Card: Red_Nine

Your Hand: Red_Six , Yellow_One , Wild_Wild_Four , Green_Eight , Yellow_Reverse , Green_DrawTwo , Blue_One ,
=====
Enter your move (play/draw): play
Enter the card to play: Red_Six
Please 'Enter' to continue

```

Figure 2 Output (Game Started)

- Displays the name of the current player's turn.
- Displays the top card of the stockpile.
- Displays the current user's deck (his hand)
- Prompts the user to draw or play a card.

```

UNO Game Started!

Current Player: Ahmed

Top Card: Red_Six

Your Hand: Green_Nine , Blue_Five , Blue_Seven , Red_DrawTwo , Yellow_Four , Yellow_Five , Green_Reverse ,
=====
Enter your move (play/draw): play
Enter the card to play: Red_DrawTwo
Please 'Enter' to continue

```

Figure 3 Output (Next player's turn)

- Next player's turn.

```

UNO Game Started!

Current Player: Fatima

Top Card: Red_DrawTwo

Your Hand: Green_Skip , Red_Nine , Yellow_One , Wild_Wild_Four , Green_Eight , Yellow_Reverse , Green_DrawTwo , Blue_One ,
=====
Enter your move (play/draw): play
Enter the card to play: Red_Nine
Please 'Enter' to continue

```

Figure 4 Output (Playing a Wild Card)

- Again, his deck is being displayed.
- On playing Red_DrawTwo, the user has two cards added to his deck as shown.

```

UNO Game Started!

Current Player: Ahmed

Top Card: Red_Nine

Your Hand: Green_Nine , Blue_Five , Blue_Seven , Yellow_Four , Yellow_Five , Green_Reverse ,
=====
Enter your move (play/draw): draw
Please 'Enter' to continue

```

Figure 5 Output (Draw Card)

- Now, the user draws instead of playing a card.

```

UNO Game Started!

Current Player: Fatima

Top Card: Red_Skip

Your Hand: Green_Skip , Yellow_One , Wild_Wild_Four , Green_Eight , Yellow_Reverse , Green_DrawTwo , Blue_One ,

=====
Enter your move (play/draw): play
Enter the card to play: Wild_Wild_Four
Enter the declared color (Red/Blue/Green/Yellow): Red
Please 'Enter' to continue

```

Figure 6 Output (Playing Wild Four)

- Now, the user has played Wild_Wild_Four which gives an additional feature to declare the next color.

```

UNO Game Started!

Current Player: Ahmed

Top Card: Green_Seven

Your Hand: Green_DrawTwo , Red_One , Red_Eight , Red_DrawTwo , Green_Nine , Wild_Wild_Four , Blue_Skip ,

=====
Enter your move (play/draw): play
Enter the card to play: Red_One
Invalid player move, expected color: Green but got color Red
Error: Invalid color submission
Please 'Enter' to continue

```

Figure 7 Output (Error Handling for Invalid Move)

- In case the user play an invalid move, error handling is done and the user is prompted to play a valid move.
- **Difficulties encountered:**
 - **Segmentation Errors:** As all the cards in the deck are being initialized in dynamic memory, the segmentation errors are quite common to occur.
 - **Stack Overflow:** It is very common for the program to abort due to stack overflow as there is a large number of nested loops and functions.
 - **Error Handling:** Due to stack overflow, it was difficult to implement try and catch blocks for each and every error as there are a long list of errors that can occur and that consumes a big chunk of static memory.
 - **GUI Implementation:** GUI Implementation became extremely difficult (hence why the proposed idea of a GUI was discarded) as the cards are being initialized in dynamic memory and assigning a card photo in dynamic memory was difficult. If arrays were being used, the GUI implementation would've been possible.
 - **The no. of players:** Although using vectors instead of arrays for the PlayerDeck array might have been a better choice but we did not go for that because we were encountering more heap errors and would have to use more exception handling to overcome that, hence we stuck with arrays and predefined the number of players in code.

- **Algorithm Issues:**

- The initial sketch of the code employed inheritance; however, each class represents a different entity and has different functionalities in the game, therefore, inheritance would not have been a better choice to make.
- First, we thought of declaring the color and value just as string attributes of Uno Card, which would have been great for inputs but that was a little difficult in predefining those attributes, since we have to have a whole deck pre-initialized as the game commences. After doing some research, we came to the conclusion that using enums is a way better choice as the attributes of Uno Card since they already get assigned integer values.
- Initialize deck was called after Uno Game object was created which resulted in error of “No card in deck”, hence we resolved that by adding the initialize deck in the start function of Uno Game class.

- **Conclusion:**

In summary, the Uno Game implementation showcases a well-organized and modular approach to creating a console-based Uno card game. The UnoGame class oversees game flow, while the UnoCard and UnoDeck classes encapsulate card-related information and deck management operations, respectively. The code successfully enforces Uno game rules, handles player interactions, and incorporates error handling for a robust gaming experience. The console interface provides clear feedback on the game state, and the implementation is designed with extensibility in mind. While achieving its primary goal, the project leaves room for future enhancements, making it a solid foundation for further exploration of graphical interfaces, additional features, and multiplayer capabilities.